

SE LAB -03

Name:- ESHWAR

SRN:- PES1UG24CS663

Section:- K

Component Modelling & Architectural Pattern Selection

Architectural Choice

We chose the Layered Architecture style for the Self-Service Coffee Kiosk System. The system is divided into three layers:

- Presentation Layer: Touch Screen UI Component
- Business Layer: Order Manager, Payment Service, and Receipt Printer Components
- Data Layer: Menu Database Component

Each layer communicates only with the layer directly below it using clearly defined interfaces: IOrderService, IPaymentService, IReceiptService, and IMenuData. The Order Manager acts as the central coordinator for the complete ordering process.

Two Reasons for the Choice

1. Suitable for a small, sequential kiosk workflow

The kiosk is a small, single-device system with a simple and fixed workflow. A customer selects one of three drinks and one of two sizes, makes a card payment, and receives a receipt. This straightforward process fits naturally into a layered structure, with the Order Manager handling the main workflow.

Using Microservices would introduce unnecessary overhead such as service discovery, network communication, and separate deployment for a system with only six menu combinations running on a single machine. Similarly, a Client-Server architecture is unnecessary because the kiosk operates independently and does not require a central server serving multiple clients.

2. Provides better separation and maintainability

Layering separates components that are more likely to change. The touch-screen interface, receipt printer, and card-payment provider may depend on specific hardware or external vendors. Since these components communicate through interfaces, one implementation can be replaced without requiring major changes to the rest of the system. For example, a new screen design, a different printer driver, or another payment provider can be introduced independently. Similarly, changes to drinks or prices can be handled within the Data layer without modifying the presentation or business logic. Each component can also be tested separately using stubs or mock implementations.

Security Advantage

The layered structure provides controlled access between components. The Touch Screen UI does not directly handle card information or access the database. All requests are routed through the Order Manager, while only the Payment Service is responsible for processing card details.

The Order Manager is also the only component that accesses the Data layer. Therefore, even if the UI is compromised or modified, it cannot directly access or alter menu prices.

This design keeps card-processing responsibilities within a single component, centralizes input validation, and reduces the scope of payment-related security requirements such as PCI-DSS compliance. Receipts and stored orders retain only masked card information rather than complete card details.

Performance Benefit

All three layers operate within the same process on the kiosk. Communication between components therefore occurs through in-process function calls, without network communication or serialization overhead. This helps keep the touch interface responsive compared with a microservices-based solution where requests would involve network calls.

Since the menu is small and changes infrequently, the Order Manager can load the menu once through IMenuData and cache it. This avoids repeatedly querying the database every time the customer interacts with the screen.

Trade-off Accepted

A layered architecture can create tighter dependencies between layers and makes it more difficult to scale individual components independently. However, this limitation is acceptable because each kiosk serves one customer at a time and operates as a standalone device. If the system is later expanded to manage many kiosks centrally, a server-side tier can be added while retaining the layered structure within each kiosk.

Software Architecture - Component Diagram

